TCP-Lab 讲义1：RDT-1.0

本讲义对应 `TCP-Lab讲义0` 的 **Step 1：跑通最小闭环（不抗差错）**，目标是在 `eFlag=0`（信道无差错/无丢包/无延迟）下，让系统能"发 → 收 → 回ACK → 写入 `recvData.txt`"，为后续 RDT2/3、滑动窗口打地基。

---

1. RDT1.0 具体是什么?

在本实验语境里，**RDT1.0 = 假设信道可靠（`eFlag=0`）的可靠数据传输最小闭环**，它不解决任何真实网络问题，只做"把链路跑通"：

- **信道假设**：不出错、不丢包、不乱序、不延迟（`eFlag=0`）。
- **发送端语义**：打包一个数据段 → 通过信道发送 → 等到接收端的 ACK（确认号等于该段 `seq`）后返回。
- **接收端语义**：收到数据段 → 直接交付（入队、写文件）→ 回 ACK(`ack = seq`)。
- **不做的事**：校验和、差错处理、超时重传、重复包处理、滑动窗口等（这些属于后续 RDT2/RDT3/GBN/SR）。

1.1 `eFlag` 是什么?

`eFlag` 不是标准 TCP 字段，而是本实验平台为了"可控地制造差错/丢包/延迟"在 `TCP_HEADER` 里额外加的控制位（字段名 `th_eflag`）。

它的作用是：**告诉模拟信道（转发服务器 `Server`）用哪种方式来"折腾"这个分组。**平台会对数据/ACK 分组以较低概率注入一次传输异常（源码里是 `rand.nextInt(100)==1`，约 1%），握手报文（SYN / SYN+ACK）不注入异常直接转发。

在 `TCP_TestCode/com/ouc/tcp/server/Server.java` 里可以看到注入逻辑（`processTransErr`）：

- 差错（corrupt）：把数据段里部分整数改成负数；若是纯 ACK 则把 `ack` 改成负数
- 丢包（loss）：直接不转发该分组
- 延迟（delay）：转发但会延迟一段时间（由 `ForwardDelay` 实现）

`th_eflag` 的取值与含义（按 `Server.listenNewPacket` 的分支）：

- `0`：不引入异常（Step 1 就用这个）
- `1`：只可能"差错"
- `2`：只可能"丢包"
- `3`：只可能"延迟"
- `4`：差错 或 丢包（随机二选一）
- `5`：差错 或 延迟（随机二选一）
- `6`：丢包 或 延迟（随机二选一）
- `7`（以及其它未列出值）：差错/丢包/延迟（随机三选一）

所以 Step 1 要"跑通最小闭环（不抗差错）"，关键就是在发送数据段和回复 ACK 时都固定 `eFlag=0`（本实现已在 `udt_send` / `reply` 中设置）。

---

2. 实验平台里闭环“靠什么连起来”？

`TCP_TestSys_Linux.jar` 提供了平台框架（`Server/Client/ListenPacket/ListenACK/TCP_HEADER/TCP_PACKET/...`）。你的代码只需要实现：

- `com.ouc.tcp.test.TCP_Sender`：实现 `rdt_send/udt_send/recv/waitACK`
- `com.ouc.tcp.test.TCP_Receiver`：实现 `rdt_recv/reply/deliver_data`

`TestRun` 启动入口是 `com.ouc.tcp.test.TestRun`，它调用 `com.ouc.tcp.app.SystemStart` 去启动：

1. 转发服务器（模拟信道）
2. Receiver（监听数据到达）
3. App_Sender（读取 `ENCDA.tcp`，分组调用 `TCP_Sender.rdt_send`）

---

3. 我做了什么（让 `TCP_Test_final` 能跑通 Step 1）？

3.1 补齐/修正了可运行所需依赖

`TCP_Test_final` 需要依赖平台 jar 才能编译运行。我将 Maven 依赖路径指向仓库根目录下的 `TCP_TestSys_Linux.jar`：

- `TCP_Test_final/pom.xml`
  - 将 `<systemPath>` 改为 `${basedir}/../TCP_TestSys_Linux.jar`

这样在不复制 jar 到 `TCP_Test_final/lib/` 的情况下，也能直接 `mvn package` 编译运行。

3.2 按“最小闭环”实现 Sender/Receiver（RDT1.0）

- `TCP_Test_final/src/com/ouc/tcp/test/TCP_Sender.java`
  - `rdt_send`：设置 `seq = dataIndex * 100 + 1`（实验约定）、封装数据段、`eFlag=0` 发送、等待对应 ACK
  - `waitACK`：从 `ackQueue` 阻塞等待，直到收到期望 `ack == seq`
- `TCP_Test_final/src/com/ouc/tcp/test/TCP_Receiver.java`
  - `rdt_recv`：收到段后入队、回 ACK(`ack=seq`)、每收满 20 组交付写入 `recvData.txt`
  - `reply`：固定 `eFlag=0` 发送 ACK

说明：RDT1.0 阶段**不需要校验和**。当前 `TCP_Test_final/src/com/ouc/tcp/test/CheckSum.java` 的 `computeChkSum()` 还是空实现（返回 0），因此报文 `th_sum` 在 Step 1 实际等价于“始终为 0”；后续做到 RDT2.0 才需要把它真正实现并接入逻辑。

---

4. 如何运行与验证（只测 eFlag=0）

验证点（Step 1 的验收口径）

- `TCP_Test_final/recvData.txt`：已生成且持续增长（说明 Receiver 已交付数据）
- `TCP_Test_final/Log.txt`：出现连续的 ACK 记录（例如 `ACK_ack: 101/201/301/...`），整体表现为“全部 ACKed”
- 可用可视化工具 `OUC-TCP-Lab-Visualizer` 打开/分析 `Log.txt`（可选）

---

5. 代码逐函数讲解（当前 `TCP_Test_final` 的实现）

这一节按“函数在做什么/由谁调用/与谁配合”来解释，目的是让你能把 Step 1 的闭环在脑子里跑通。

5.1 `TCP_Sender`：发送端（停等 + 等 ACK）

文件：`TCP_Test_final/src/com/ouc/tcp/test/TCP_Sender.java`

成员变量

- `tcpPack`：当前“正在等待 ACK 的那一个数据报文”（停等协议里一次只在飞一个）。
- `flag`：发送线程与 ACK 监听线程之间的共享标志位。
  - `0`：还没收到期望 ACK（`rdt_send` 会一直等）
  - `1`：已收到期望 ACK（`rdt_send` 返回）

此外，父类 `TCP_Sender_ADT` 里还有关键成员（你在子类里直接用到了）：

- `client`：UDP socket 封装，用来 `send()`
- `tcpH` / `tcpS`：可复用的 TCP 头部对象/数据段对象（每次发送前填字段）
- `destinAddr` / `destinPort`：接收端地址端口
- `ackQueue`：ACK 队列（`ListenACK` 线程收到 ACK 会回调 `recv()`，由你把 ACK 号塞进来）

`public TCP_Sender()`

做两件事：

1. `super()`：创建底层 `Client` 并初始化 `ackQueue` 等；打印 Sender socket 地址
2. `initTCP_Sender(this)`：启动 `ListenACK` 监听线程，并初始化默认的 `tcpH/tcpS`

也就是说：**Sender 一创建，就开始“后台收 ACK”。**

`public void rdt_send(int dataIndex, int[] appData)`

这是“应用层调用的可靠发送接口”，Step 1 的闭环就从这里开始。

它做的事按顺序是：

1. 计算并设置序号：`th_seq = dataIndex * appData.length + 1`
   - `App_Sender` 每次传 100 个整数（`appData.length=100`），所以 `seq` 是 1、101、201…

2. 把数据写入段：`tcpS.setData(appData)`
3. 组装成一个完整报文：`tcpPack = new TCP_PACKET(tcpH, tcpS, destinAddr)`
4. 填校验和字段：`th_sum = CheckSum.computeChkSum(tcpPack)`
    - Step 1 里该函数返回 0，所以 `th_sum` 实际一直是 0
5. 调 `udt_send(tcpPack)` 发出去（注意：`udt_send` 会把 `eFlag` 固定成 0）
6. 把 `flag` 清 0，进入等待：`while(flag==0);`
    - **这是"停等"的等 ACK 阶段**：只有当后台收到 ACK 并把 `flag` 改成 1，这里才会返回

你可以把它理解成："发一个包，然后阻塞等它被确认"。

`public void udt_send(TCP_PACKET stcpPack)`

这是"通过不可靠信道发送"的接口（名字叫 UDT），但 Step 1 我们把信道配置成可靠：

1. 设置 `eFlag=0`：告诉模拟信道不要注入差错/丢包/延迟
2. `client.send(stcpPack)`：真正把对象序列化成 UDP 报文发出去

`public void recv(TCP_PACKET recvPack)`

这是**ACK 到达后的回调函数**（由父类启动的 `ListenACK` 线程调用）。

它做的事是：

1. 从 ACK 报文里取出确认号 `th_ack`
2. 放入 `ackQueue`
3. 立刻调用 `waitACK()` 去处理这个 ACK（决定是否让 `rdt_send` 放行）

注意：`rdt_send` 本身在忙等 `flag`，它不主动读队列；所以这里必须在收到 ACK 时触发处理。

`public void waitACK()`

这个函数的任务是：**从 `ackQueue` 取一个 ACK 号，判断它是不是当前 `tcpPack` 的确认。**

当前代码逻辑是：

- 如果队列非空：取出一个 ACK 号 `currentAck`
    - 若 `currentAck == tcpPack.seq`：说明确认了"当前正在等的包"，把 `flag=1`，发送线程就会结束等待
    - 否则：认为 ACK 不匹配（或是 NACK=-1），调用 `udt_send(tcpPack)` 立刻重传，并保持 `flag=0`

在 Step 1（`eFlag=0`）下基本不会走到"重传"，因为 ACK 不会出错/丢失。

5.2 `TCP_Receiver`：接收端（收数据 + 回 ACK + 写文件）

文件：`TCP_Test_final/src/com/ouc/tcp/test/TCP_Receiver.java`

成员变量

- `ackPack`：即将发回去的 ACK 报文
- `sequence`：用于打印/调试的"期望序号"计数（当前代码并未用它做丢包/乱序判断）

父类 `TCP_Receiver_ADT` 里还有关键成员：

- `client`：UDP socket 封装，用来 `send()`
- `tcpH/tcpS`：可复用的 TCP 头部对象/数据段对象
- `dataQueue`：接收端交付队列（收够一定量就写 `recvData.txt`）

`public TCP_Receiver()`

做两件事：

1. `super()`：创建底层 `Client`、初始化 `dataQueue`，并把 `recvData.txt` 清空
2. `initTCP_Receiver(this)`：启动 `ListenPacket` 监听线程，并初始化默认的 `tcpH/tcpS`

也就是说：**Receiver 一创建，就开始"后台收数据包"。**

`public void rdt_recv(TCP_PACKET recvPack)`

这是"可靠接收入口"（由 `ListenPacket` 线程在收到数据包后回调）。

它做的事是：

1. 校验：`computeChkSum(recvPack) == recvPack.th_sum`
    - Step 1 下由于两边都填 0，所以会通过
2. 通过校验时（认为正确包）：
    - 生成 ACK：`th_ack = recvPack.th_seq`
    - 构造 `ackPack = new TCP_PACKET(tcpH, tcpS, recvPack.getSourceAddr())`
    - 设置 ACK 报文校验和（Step1 仍为 0）
    - `reply(ackPack)` 发回 ACK
    - 把数据段 `recvPack.tcpS.data` 放进 `dataQueue`
    - `sequence++`（仅用于调试输出）
3. 未通过校验时：
    - 打印一些调试信息
    - 设置 `th_ack = -1` 作为 NACK（否认确认）
    - 回发这个 ACK/NACK（Step 1 基本不会触发）
4. 每当 `dataQueue.size() == 20`：调用 `deliver_data()` 把累计的数据写入文件

`public void deliver_data()`

交付函数：把 `dataQueue` 里的 int 数组按行追加写入 `recvData.txt`。

关键点：

- 以"追加模式"打开文件：`new FileWriter(fw, true)`

- 每次从队列 `poll()` 一个 `int[]`，把数组里的每个整数写成一行
- 写完 `flush()`，最后 `close()`

为什么是 “20 组一次”？这是实验平台/讲义里为了减少频繁 IO 的一个节奏控制，方便观察“批量交付”。

`public void reply(TCP_PACKET replyPack)`

把 ACK 报文发回给 Sender：

1. 固定 `eFlag=0`（Step 1 不注入异常）
2. `client.send(replyPack)`

---

5. 从 RDT1.0 继续往后怎么走（提示）

按 `docs/讲义-TCP实验实现顺序与通俗讲解.md` 的路线：

1. RDT2.0：实现 `CheckSum.computeChkSum()`，Receiver 校验失败时拒收/回 -1（或重复 ACK），Sender 据此重传
2. RDT3.0：加入超时计时器，处理丢包/延迟（`eFlag=2/3/6/7`）
3. 再做滑动窗口（GBN/SR）与拥塞控制（Tahoe/Reno）